Rovel: AWS EC2 Production Deployment Guide

This guide provides a sequential, step-by-step walkthrough to deploy the Rovel PaaS platform on an AWS EC2 instance. Each step is documented with the commands used, their meaning, and the exact permissions and configuration adjustments needed beforehand to prevent errors.

Core Architecture Overview

Rovel is a self-hosted Platform-as-a-Service (PaaS). It runs a Next.js web dashboard (control plane) and a Node.js worker daemon. The worker compiles and deploys developer applications in isolated Docker containers, dynamically routing traffic to them via Nginx reverse proxies, managing Scale-to-Zero hibernation, intercepting cold starts with an Auto-Wake Gateway, and securing them with Let's Encrypt SSL certificates.

Step 1: AWS EC2 Instance Provisioning

In your AWS EC2 Console, configure the following settings to launch your virtual server:

- Name: rovel-server
- Operating System (AMI): Select Ubuntu Server 24.04 LTS (HVM).
- *Reason*: The automated bootstrap script (setup.sh) is designed for Debian/Ubuntu environments (apt-get). It will fail on Amazon Linux, RHEL, or SUSE.
- Instance Type: Select t3.small (2 vCPUs, 2 GiB RAM).
- *Reason*: Compiling Next.js, React, or Python applications during container builds consumes significant memory. While t3.micro (1 GB RAM) can be used with large swap files, t3.small offers a much more stable and faster build experience.
- Key Pair: Create or select an RSA key pair in .pem format (e.g., rovel-key.pem).
- Storage: Set the root volume size to 30 GiB (type gp3).
- *Reason*: Docker image layers and build caches occupy substantial disk space. AWS provides up to 30 GB of SSD storage for free under the 12-Month Free Tier.

Network Security Groups

Configure your security group rules to allow the following public ingress traffic:

Port	Protocol	Source	Purpose
22	TCP (SSH)	Anywhere (`0.0.0.0/0` or My IP)	Remote terminal access
80	TCP (HTTP)	Anywhere (`0.0.0.0/0`)	Domain validation, HTTP routing, and SSL r
443	TCP (HTTPS)	Anywhere (`0.0.0.0/0`)	Secure SSL dashboard and user application

---

Step 2: DNS & Domain Configuration

Go to your DNS registrar (e.g., Name.com, Route 53) and point your domain records to the Public IPv4 address of your EC2 instance (e.g., 13.204.53.200). Create the following three A records:

Type	Host / Name	Answer / Value	Purpose
A	`rovel.dev` (or `@`)	`YOUR_EC2_PUBLIC_IP`	Main landing website
A	`console`	`YOUR_EC2_PUBLIC_IP`	Developer dashboard console

A	`*.apps`	`YOUR_EC2_PUBLIC_IP`	Wildcard routing for deployed developer app

Step 3: Server Connection & Key Permissions

1. Restrict Key Permissions (Windows PowerShell)

Windows SSH client requires that your private key file (.pem) is only accessible by your current user account. Navigate to the folder containing your key (usually ~\Downloads) and run:

```
icacls.exe rovel-key.pem /grant:r "$($env:USERNAME):(R)"
icacls.exe rovel-key.pem /inheritance:r
```

Reason: If key permissions are too open, the SSH connection will be rejected with an "Unprotected Private Key File" error.

2. Connect via SSH

```
ssh -i rovel-key.pem ubuntu@YOUR_EC2_PUBLIC_IP
```

Step 4: Configure Virtual Memory (Swap Space)

Even on t3.small (2 GB RAM), container builds can spike memory usage and trigger the Out-Of-Memory (OOM) killer. Set up a 4 GB swap file on your SSD:

```
# Create a 4 GB blank file
sudo fallocate -l 4G /swapfile
# Restrict file access to root
sudo chmod 600 /swapfile
# Format it as swap space
sudo mkswap /swapfile
# Enable the swap space
sudo swapon /swapfile
# Persist across system reboots
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

Verify with free -h. You should see Swap: 4.0Gi.

Step 5: Adjust Directory Permissions (Critical)

Before running the initial build, configure the permissions for the Nginx and Let's Encrypt directories:

1. Let's Encrypt Traversal Permissions

```
sudo chmod -R 755 /etc/letsencrypt/
```

Reason: Let's Encrypt certificates are symlinked from /etc/letsencrypt/live/ to /etc/letsencrypt/archive/. If parent directories lack 755 permissions, the non-root ubuntu worker process cannot verify certificate existence and will fall back to HTTP.

2. Nginx Sites Directory Ownership

```
sudo chown -R ubuntu:ubuntu /etc/nginx/sites-enabled/
```

Reason: When a new app is deployed, the worker generates and writes an Nginx .conf file into /etc/nginx/sites-enabled/. If owned by root, the build will fail with a Permission Denied (EACCES) error.

Step 6: Clone and Bootstrap Rovel

Clone the repository into the ubuntu home directory and run the setup script:

```
cd ~
git clone https://github.com/atharvabaodhankar/Rovel.git
cd Rovel
chmod +x setup.sh update.sh
./setup.sh
```

Step 7: Environment Configuration

Create and configure your production environment file in the root ~/Rovel folder:

```
nano .env.production
```

Paste your production configuration. Ensure the following values are configured:

```
# Database Credentials
DATABASE_URL="postgresql://postgres:YOUR_PASSWORD@localhost:5432/rovel?schema=public"
REDIS_URL="redis://localhost:6379"
DB_USER=postgres
DB_PASSWORD=YOUR_PASSWORD
DB_NAME=rovel

# Domain Configuration
BASE_DOMAIN="apps.rovel.dev"
NEXT_PUBLIC_BASE_DOMAIN="apps.rovel.dev"

# Security (32-byte hex keys generated using openssl)
JWT_SECRET="YOUR_JWT_SECRET"
ENCRYPTION_KEY="YOUR_32_CHAR_ENCRYPTION_KEY"

# GitHub OAuth App (Registered for console.rovel.dev)
GITHUB_CLIENT_ID="Ov23lipY6oaKaX3j80hg"
GITHUB_CLIENT_SECRET="YOUR_GITHUB_OAUTH_SECRET"
```

Step 8: Boot Services and Compile Code

Run these commands to start the databases, install packages, sync schemas, and compile the platform:

```
# 1. Start Postgres and Redis containers
docker compose up -d

# 2. Install application dependencies
npm install
```

```
# 3. Push database schemas
node -e "const fs = require('fs'); const path = require('path'); const envPath = fs.existsSync('.env.production') ? '.env.production' : '.env'; require('dotenv').config({ path: envPath }); require('child_process').spawn('npm', ['run', 'db:push', '-w', 'packages/db'], { stdio: 'inherit', shell: true })"

# 4. Compile Next.js and worker code
npm run build:all
```

Step 9: Configure Nginx Routing

Copy the server block configuration and remove the default index page:

```
sudo cp infrastructure/nginx/rovel.conf /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
```

Open /etc/nginx/sites-enabled/rovel.conf and verify server_name includes both domains:

```
server_name console.rovel.dev rovel.dev;
```

Reload Nginx:

```
sudo systemctl reload nginx
```

Step 10: Generate SSL Certificates

1. Provision SSL for the Dashboard and Root Domain

```
sudo certbot --nginx -d console.rovel.dev -d rovel.dev
```

2. Generate the Wildcard SSL Certificate (For User Applications)

```
sudo certbot certonly --manual --preferred-challenges=dns -d "*.apps.rovel.dev" -d "apps.rovel.dev"
```

1. Add the _acme-challenge.apps TXT record in your registrar.
2. Wait 60 seconds for DNS propagation, then press Enter in your terminal.

Step 11: Start Services in PM2

```
# Install PM2 globally
sudo npm install -g pm2

# Start the dashboard and worker
pm2 start npm --name "rovel-dashboard" -- run start -w apps/web
pm2 start dist/index.js --name "rovel-worker" --cwd apps/worker

# Configure PM2 to restart automatically on system reboots
pm2 startup systemd
```

```
# (Copy and execute the output command from terminal)
pm2 save
```

Step 12: Scale-to-Zero & Auto-Wake Gateway

Rovel includes built-in container hibernation to save RAM on your EC2 instance:

- Automatic Reaper: The worker process checks containers every 60 seconds. Inactive containers are stopped with docker stop after their configured idle timeout (default: 15 minutes).

- Nginx 502 Interceptor: When a user visits a sleeping app, Nginx intercepts the 502 Bad Gateway error (error_page 502 503 504 = @waking_page;) and proxies to /wake?app=<slug>.

- Auto-Wake Gateway: The visitor sees an animated dark-mode loading UI while POST /api/wake runs docker start in the background, automatically refreshing into the live application in < 2 seconds.

Step 13: Branch Deployments & Rollbacks

- Branch Builds: Deploy any branch from the dashboard or via GitHub push webhooks.

- Badges: Pushes to the default branch are tagged Live Production; other branches are tagged Preview.

- 1-Click Rollback / Promote: On any ready deployment, click "Promote" in the Global Deployments table or Project History tab to instantly swap the running container image to that version.

Step 14: Automated CI/CD (GitHub Actions)

Add the following GitHub Secrets to your repository (Settings -> Secrets and variables -> Actions):

- EC2_HOST: YOUR_EC2_PUBLIC_IP

- EC2_USER: ubuntu

- EC2_SSH_KEY: The contents of your rovel-key.pem private key.

Every git push origin main will automatically run [update.sh](file:///c:/Users/baodh/OneDrive/Desktop/Projects/CodeShip/update.sh) to pull code, run Prisma migrations, rebuild workspaces, and restart PM2 without manual SSH.